

杭州电子科技大学

实验报告

课程名称：密码学基础与算法课程设计 姓名：xx 学号：xx

实验地点：4 教 441-443-445-447

实验时间：周五 3-5 节

一、实验名称：SM4 密码实验

二、实验要求：

- 1、熟悉分组密码的基本框架。
- 2、掌握 SM4 密码的加解密原理。
- 3、用合适的编程语言实现 SM4 密码程序并输出结果。

三、实验内容：

SM4 算法是我国商用密码标准，其前身是 SMS4 算法。SM4 算法是一个分组加密算法，分组长度和密钥长度均 128bit。SM4 算法使用 32 轮的非线性迭代结构。SM4 在最后一轮非线性迭代之后加上了一个反序变换，因此 SM4 中只要解密密钥是加密密钥的逆序，它的解密算法与加密算法就可以保持一致。SM4 的主体运算是非平衡 Feistel 网络。整体逻辑结构如图 1 所示，经过 32 轮变换把明文变换为密文。

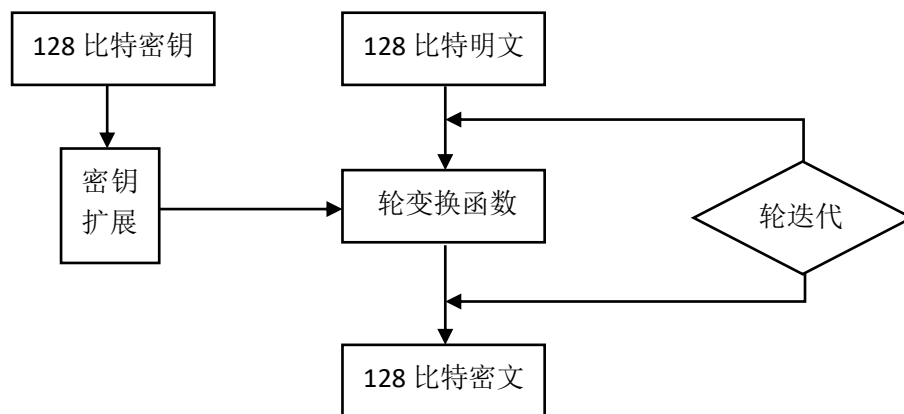


图 1 SM4 算法总体流程

其中密钥扩展运算把 128 bit 的种子密钥扩展为 32 个 32 bit 的子密钥。下面分别介绍轮函数、密钥扩展和加解密。

1. 轮函数

轮函数的规则由 $X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus RK_i)$ 给出，其中 $i = 0, 1, \dots, 31$ 。第 i 轮的输入为 $(X_i, X_{i+1}, X_{i+2}, X_{i+3})$ ，输出为 $(X_{i+1}, X_{i+2}, X_{i+3}, X_{i+4})$ 。第一轮输入 (X_0, X_1, X_2, X_3) ，即为 128 bit 明文的四个分组，最后一轮的输出 $(X_{32}, X_{33}, X_{34}, X_{35})$ ，再经过逆序，得到了密文 $(X_{35}, X_{34}, X_{33}, X_{32})$ 。下图即为轮函数的结构。

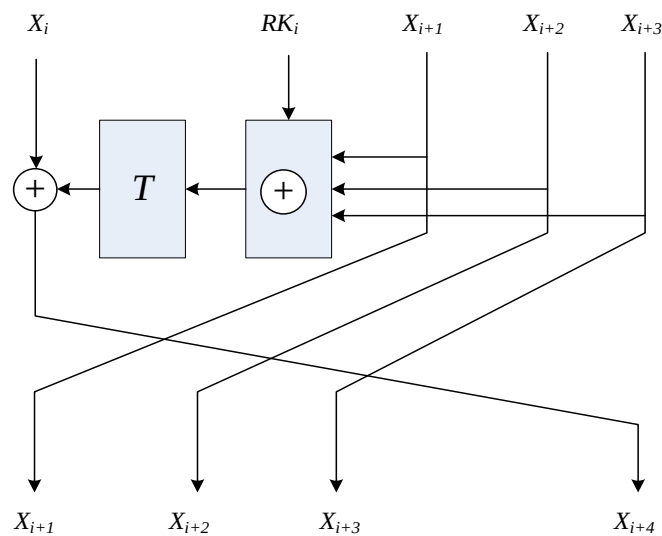


图 2 SM4 算法轮函数

SM4 算法的合成置换 T 是 $F_2^{32} \rightarrow F_2^{32}$ 的可逆置换。 T 置换是由一个非线性变换 τ 和一个线性扩散变换 L 复合而成，即 $T(\cdot) = L(\tau(\cdot))$ 。 T 置换的过程如下图所示：

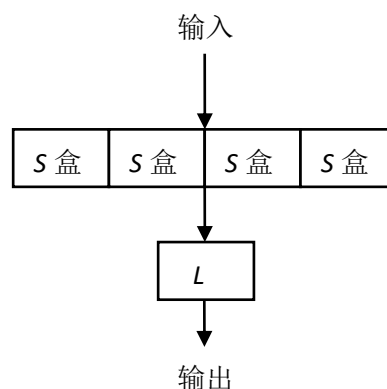


图 3 SM4 算法 L 函数

非线性变换 τ 由四个 S 盒并行组成。设变换 τ 的输入 $A = (a_0, a_1, a_2, a_3) \in (F_2^8)^4$ ，

输出是 $B = (b_0, b_1, b_2, b_3) \in (F_2^8)^4$ ，则 $(b_0, b_1, b_2, b_3) = (S(a_0), S(a_1), S(a_2), S(a_3))$ 。不同于 DES 等分组密码算法，SM4 算法中的这四个 S 盒实际上是同一个 8 bit -> 8bit 的 S 盒，详见下表：

表 1 SM4 算法的 S 盒

	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xa	0xb	0xc	0xd	0xe	0xf
0x0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
0x1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
0x2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
0x3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
0x4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
0x5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
0x6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
0x7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
0x8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
0x9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
0xa	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
0xb	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
0xc	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
0xd	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
0xe	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
0xf	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

其中左边的列表示 8 bit 输入的高位部分，上方的行表示 8 bit 输入的低位部分。

非线性变换 τ 的输出是线性变换 L 的输入，设 L 的输入为 $B \in F_2^{32}$ ，输出为 $C \in F_2^{32}$ ，则

$$C = L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

其中 $B \lll n$ 表示 B 循环左移 n 位。

2. 密钥扩展

在密钥扩展方案中，种子密钥经过扩展生成 32 个轮密钥，每个轮密钥长度为 32 bit。首先，128 bit 的种子密钥 SK 分为四组 $SK = (SK_0, SK_1, SK_2, SK_3) \in (F_2^8)^4$ ，再给定系统参数

$$FK = (FK_0, FK_1, FK_2, FK_3) = \{0xa3b1bac6, 0x56aa3350, 0x677d9197, 0xb27022dc\};$$

与固定参数 $CK_i = (ck_{i0}, ck_{i1}, ck_{i2}, ck_{i3}) \in (F_2^8)^4$ (其中 $ck_{ij} = 7(4i + j) \bmod 256$)，密钥扩展规则如下：

$$(K_0, K_1, K_2, K_3) = (SK_0 \oplus FK_0, SK_1 \oplus FK_1, SK_2 \oplus FK_2, SK_3 \oplus FK_3)$$
$$RK_i = K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

其中 $i = 0, 1, 2, \dots, 31$ 用于生成共 32 个轮密钥。 T' 变换与加密算法轮函数中的变换除线性变换 L 不同外，其他相同。 T' 变换中的线性变换 L' 为

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

3. 加解密

加密与解密的轮函数结构完全相同，唯一的区别是解密密钥是加密密钥的逆序。加密轮密钥的使用顺序为 $RK_0, \dots, RK_{31}$ ，解密时轮密钥的使用顺序为 $RK_{31}, \dots, RK_0$ 。

4. 主要步骤

- 1) 新建一个空项目，取名 sm4_test。
- 2) 在左边的解决方案资源管理器中添加 cpp 文件，取名为 sm4_test.cpp。
- 3) 在 SM4 中，32bit 可用一个无符号整型(unsigned int)变量表示，128bit 可用长度为 4 的无符号整型数组表示。
- 4) 在 sm4_test.cpp 中先写入

```
#include<iostream>

using namespace std;

static unsigned int RK[32] = {0};    //32 轮轮密钥, 每个 RKi 为 32bit

static unsigned int K[36] = {0};    //在密钥扩展中使用, 一共 36 组, 每个 Ki 为 32bit

static const unsigned char S_Box[256] = {    //S 盒, 输入 8bit, 输出 8bit

0xd6,0x90,0xe9,0xfe,0xcc,0xe1,0x3d,0xb7,0x16,0xb6,0x14,0xc2,0x28,0xfb,0x2c,0x05,

0x2b,0x67,0x9a,0x76,0x2a,0xbe,0x04,0xc3,0xaa,0x44,0x13,0x26,0x49,0x86,0x06,0x99,
```

```

0x9c,0x42,0x50,0xf4,0x91,0xef,0x98,0x7a,0x33,0x54,0x0b,0x43,0xed,0xcf,0xac,0x62,
0xe4,0xb3,0x1c,0xa9,0xc9,0x08,0xe8,0x95,0x80,0xdf,0x94,0xfa,0x75,0x8f,0x3f,0xa6,
0x47,0x07,0xa7,0xfc,0xf3,0x73,0x17,0xba,0x83,0x59,0x3c,0x19,0xe6,0x85,0x4f,0xa8,
0x68,0x6b,0x81,0xb2,0x71,0x64,0xda,0x8b,0xf8,0xeb,0x0f,0x4b,0x70,0x56,0x9d,0x35,
0x1e,0x24,0x0e,0x5e,0x63,0x58,0xd1,0xa2,0x25,0x22,0x7c,0x3b,0x01,0x21,0x78,0x87,
0xd4,0x00,0x46,0x57,0x9f,0xd3,0x27,0x52,0x4c,0x36,0x02,0xe7,0xa0,0xc4,0xc8,0x9e,
0xea,0xbf,0x8a,0xd2,0x40,0xc7,0x38,0xb5,0xa3,0xf7,0xf2,0xce,0xf9,0x61,0x15,0xa1,
0xe0,0xae,0x5d,0xa4,0x9b,0x34,0x1a,0x55,0xad,0x93,0x32,0x30,0xf5,0x8c,0xb1,0xe3,
0x1d,0xf6,0xe2,0x2e,0x82,0x66,0xca,0x60,0xc0,0x29,0x23,0xab,0x0d,0x53,0x4e,0x6f,
0xd5,0xdb,0x37,0x45,0xde,0xfd,0x8e,0x2f,0x03,0xff,0x6a,0x72,0x6d,0x6c,0x5b,0x51,
0x8d,0x1b,0xaf,0x92,0xbb,0xdd,0xbc,0x7f,0x11,0xd9,0x5c,0x41,0x1f,0x10,0x5a,0xd8,
0x0a,0xc1,0x31,0x88,0xa5,0xcd,0x7b,0xbd,0x2d,0x74,0xd0,0x12,0xb8,0xe5,0xb4,0xb0,
0x89,0x69,0x97,0x4a,0x0c,0x96,0x77,0x7e,0x65,0xb9,0xf1,0x09,0xc5,0x6e,0xc6,0x84,
0x18,0xf0,0x7d,0xec,0x3a,0xdc,0x4d,0x20,0x79,0xee,0x5f,0x3e,0xd7,0xcb,0x39,0x48};

```

```
static unsigned int CK[32] = {0}; //定义固定参数 CK, 32 组, 每个 CKi 为 32bit
```

```
static unsigned int FK[4] = { 0xa3b1bac6, 0x56aa3350, 0x677d9197, 0xb27022dc};
```

```
//给定的系统参数 FK, 4 组, 每个 FKi 为 32bit
```

4) 编写各个模块函数，例如：

```

unsigned int S_Func(unsigned int In); //S 函数

void SetRoundKey(unsigned int SK[]); //密钥扩展函数

unsigned int RotL(unsigned int In, int loop); //循环左移函数

unsigned int T(unsigned int In); //T 置换

unsigned int T1(unsigned int In); //T'置换

unsigned int L_Func(unsigned int In); //L 变换

unsigned int L1_Func(unsigned int In); //L'变换

void Crypt(unsigned int Out[], unsigned int In[], unsigned int K, bool flag); //加解密

void SetPara(); //设置固定参数 CK

```

5) 编写主函数，调试程序，目的是测试各个函数编写正确。SM4 的标准文档里给出以下的测试示例：

明文 = 0x0123456789abcdeffedcba9876543210 (128bit)

密钥 = 0x0123456789abcdeffedcba9876543210 (128bit)

则密文 = 0x681edf34d206965e86b3e94f536e4246 (128bit)

6) 所有函数测试都正确后，新建一个对话框项目 SM4，包含明文框、密钥框、密文框、解密文框等，以及加密，解密，清空按钮，将以上的全局变量、S 盒以及所有子函数声明及实现代码全部复制到 SM4Dlg.cpp 中的合适位置，再编写按钮事件，实现加解密功能。

部分函数参考代码：

```
unsigned int RotL(unsigned int In, int loop)
{
    return (In << loop) | (In >> (32 - loop));
}

unsigned int L_Func(unsigned int In)
{
    return In ^ RotL(In,2) ^ RotL(In,10) ^ RotL(In,18) ^ RotL(In,24);
}

unsigned int L1_Func(unsigned int In)
{
    return In ^ RotL(In,13) ^ RotL(In,23);
}

unsigned int S_Func(unsigned int In)
{
    unsigned int Out = 0;
    unsigned char temp = {0};
    for(int i = 0; i<4; i++) //4 组 8bit 的部分分别经过 S 盒, 再拼成 32bit
    {
        temp = ((In >> (24 - 8 * i)) & 0xFF);
        Out = Out + (S_Box[temp] << (24 - 8 * i));
    }
    return Out;
}

unsigned int T(unsigned int In)
{
    return L_Func(S_Func(In));
}
```

```

unsigned int T1(unsigned int In)
{
    return L1_Func(S_Func(In));
}

void SetPara() //设置所有固定参数 CKi
{
    unsigned int temp = 0;
    for(int i = 0; i<32; i++)
    {
        for(int j = 0; j <4; j++)
        {
            temp = (7 * (4 * i + j)) & 0xFF;
            temp = temp << (24 - 8 * j);
            CK[i] = CK[i] + temp;
        }
    }
}

```

SM4 密码程序代码如下：(所有模块函数代码以及按钮事件代码)

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
static unsigned int RK[32] = {0}; //32 轮轮密钥，每个 RKi 为 32bit
```

```
static unsigned int K[36] = {0}; //在密钥扩展中使用，一共 36 组，每个 Ki 为 32bit
```

```
static const unsigned char S_Box[256] = { //S 盒，输入 8bit，输出 8bit
```

```
    0xd6, 0x90, 0xe9, 0xfe, 0xcc, 0xe1, 0x3d, 0xb7, 0x16, 0xb6, 0x14, 0xc2, 0x28,
    0xfb, 0x2c, 0x05,
```

```
    0x2b, 0x67, 0x9a, 0x76, 0x2a, 0xbe, 0x04, 0xc3, 0xaa, 0x44, 0x13, 0x26, 0x49,
    0x86, 0x06, 0x99,
```

```
    0x9c, 0x42, 0x50, 0xf4, 0x91, 0xef, 0x98, 0x7a, 0x33, 0x54, 0x0b, 0x43, 0xed,
    0xcf, 0xac, 0x62,
```

```
    0xe4, 0xb3, 0x1c, 0xa9, 0xc9, 0x08, 0xe8, 0x95, 0x80, 0xdf, 0x94, 0xfa, 0x75,
    0x8f, 0x3f, 0xa6,
```

```

    0x47, 0x07, 0xa7, 0xfc, 0xf3, 0x73, 0x17, 0xba, 0x83, 0x59, 0x3c, 0x19, 0xe6,
    0x85, 0x4f, 0xa8,
    0x68, 0x6b, 0x81, 0xb2, 0x71, 0x64, 0xda, 0x8b, 0xf8, 0xeb, 0x0f, 0x4b, 0x70,
    0x56, 0x9d, 0x35,
    0x1e, 0x24, 0x0e, 0x5e, 0x63, 0x58, 0xd1, 0xa2, 0x25, 0x22, 0x7c, 0x3b, 0x01,
    0x21, 0x78, 0x87,
    0xd4, 0x00, 0x46, 0x57, 0x9f, 0xd3, 0x27, 0x52, 0x4c, 0x36, 0x02, 0xe7, 0xa0,
    0xc4, 0xc8, 0x9e,
    0xea, 0xbf, 0x8a, 0xd2, 0x40, 0xc7, 0x38, 0xb5, 0xa3, 0xf7, 0xf2, 0xce, 0xf9,
    0x61, 0x15, 0xa1,
    0xe0, 0xae, 0x5d, 0xa4, 0x9b, 0x34, 0x1a, 0x55, 0xad, 0x93, 0x32, 0x30, 0xf5,
    0x8c, 0xb1, 0xe3,
    0x1d, 0xf6, 0xe2, 0x2e, 0x82, 0x66, 0xca, 0x60, 0xc0, 0x29, 0x23, 0xab, 0x0d,
    0x53, 0x4e, 0x6f,
    0xd5, 0xdb, 0x37, 0x45, 0xde, 0xfd, 0x8e, 0x2f, 0x03, 0xff, 0x6a, 0x72, 0x6d,
    0x6c, 0x5b, 0x51,
    0x8d, 0x1b, 0xaf, 0x92, 0xbb, 0xdd, 0xbc, 0x7f, 0x11, 0xd9, 0x5c, 0x41, 0x1f,
    0x10, 0x5a, 0xd8,
    0x0a, 0xc1, 0x31, 0x88, 0xa5, 0xcd, 0x7b, 0xbd, 0x2d, 0x74, 0xd0, 0x12, 0xb8,
    0xe5, 0xb4, 0xb0,
    0x89, 0x69, 0x97, 0x4a, 0x0c, 0x96, 0x77, 0x7e, 0x65, 0xb9, 0xf1, 0x09, 0xc5,
    0x6e, 0xc6, 0x84,
    0x18, 0xf0, 0x7d, 0xec, 0x3a, 0xdc, 0x4d, 0x20, 0x79, 0xee, 0x5f, 0x3e, 0xd7,
    0xcb, 0x39, 0x48
};

static unsigned int CK[32] = {0};    //定义固定参数 CK, 32 组, 每个 CKi 为 32bit
static unsigned int FK[4] = { 0xa3b1bac6, 0x56aa3350, 0x677d9197, 0xb27022dc};
//给定的系统参数 FK, 4 组, 每个 FK_i 为 32bit

```


//函数声明

unsigned int S_Func(unsigned int In);

void SetRoundKey(unsigned int SK[]);

unsigned int RotL(unsigned int In, int loop);

unsigned int T(unsigned int In);

unsigned int T1(unsigned int In);

unsigned int L_Func(unsigned int In);

unsigned int L1_Func(unsigned int In);

void SetPara();

void Crypt(unsigned int Out[], unsigned int In[], unsigned int RK[], bool flag);

//密钥扩展函数

void SetRoundKey(unsigned int SK[])

{

 //初始化 K[0] 到 K[3] 为 SK 与 FK 的异或结果

 for (int i = 0; i < 4; i++) {

 K[i] = SK[i] ^ FK[i];

 }

 //根据密钥扩展规则生成 K[4] 到 K[35]

 for (int i = 0; i < 32; i++) {

 unsigned int temp = T1(K[i + 1] ^ K[i + 2] ^ K[i + 3] ^ CK[i]);

 //生成 K[i+4]

 K[i + 4] = K[i] ^ temp;

 //将 K[i+4] 赋给 RK[i], 用于生成轮密钥

 RK[i] = K[i + 4];

 }

```
}
```

```
//循环左移函数
```

```
unsigned int RotL(unsigned int In, int loop)
{
    return (In << loop) | (In >> (32 - loop));
}
```

```
//S 函数
```

```
unsigned int S_Func(unsigned int In)
{
    unsigned int Out = 0;
    unsigned char temp = {0};
    for (int i = 0; i < 4; i++)
    {
        temp = ((In >> (24 - 8 * i)) & 0xFF);
        Out = Out + (S_Box[temp] << (24 - 8 * i));
    }
    return Out;
}
```

```
//L 变换
```

```
unsigned int L_Func(unsigned int In)
{
    return In ^ RotL(In, 2) ^ RotL(In, 10) ^ RotL(In, 18) ^ RotL(In, 24);
}
```

```
//L'变换
```

```
unsigned int L1_Func(unsigned int In)
{
```

```

        return In ^ RotL(In, 13) ^ RotL(In, 23);
    }

//T 置换
unsigned int T(unsigned int In)
{
    return L_Func(S_Func(In));
}

//T'置换
unsigned int T1(unsigned int In)
{
    return L1_Func(S_Func(In));
}

//设置固定参数 CKi
void SetPara()
{
    unsigned int temp = 0;
    for (int i = 0; i < 32; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            temp = (7 * (4 * i + j)) & 0xFF;
            temp = temp << (24 - 8 * j);
            CK[i] = CK[i] + temp;
        }
    }
}

```

//加解密函数

```
void Crypt(unsigned int Out[], unsigned int In[], unsigned int RK[], bool flag)
{
    unsigned int X[36]; // 存储轮函数的输入/输出

    // 初始化 X[0] 到 X[3] 为明文的四个分组
    for (int i = 0; i < 4; i++)
    {
        X[i] = In[i];
    }

    if (flag)
    { // 执行 32 轮加密
        for (int i = 0; i != 32; i++) {
            unsigned int temp_val = X[i + 1] ^ X[i + 2] ^ X[i + 3] ^ RK[i];
            X[i + 4] = X[i] ^ T(temp_val);
        }
    }
    else
    {
        // 执行 32 轮解密
        for (int i = 0; i != 32; i++) {
            unsigned int temp_val = X[i + 1] ^ X[i + 2] ^ X[i + 3] ^ RK[31-i];
            X[i + 4] = X[i] ^ T(temp_val);
        }
    }

    // 逆序输出，得到密文或明文
    for (int i = 0; i < 4; i++) {
        Out[i] = X[35 - i]; // 逆序获取密文或明文
    }
}
```

```
}
```

```
int main() {  
    // 用户输入明文和密钥  
    string plaintext, key;  
    cout << "请输入明文 (ASCII 字符串): ";  
    getline(cin, plaintext); // 输入任意长度的明文  
    cout << "请输入密钥 (ASCII 字符串): ";  
    getline(cin, key); // 输入任意长度的密钥  
  
    // 输出明文的十六进制表示  
    cout << "明文的十六进制表示: ";  
    for (int i = 0; i < plaintext.length(); i++) {  
        printf("%02x ", (unsigned char)plaintext[i]);  
    }  
    cout << endl;  
  
    // 输出密钥的十六进制表示  
    cout << "密钥的十六进制表示: ";  
    for (int i = 0; i < key.length(); i++) {  
        printf("%02x ", (unsigned char)key[i]);  
    }  
    cout << endl;  
  
    // 处理输入的密钥，将其转换为 4 个 32 位整数  
    unsigned int Key[4] = {0};  
    int key_len = key.length();  
    for (int i = 0; i < 4; i++) {  
        Key[i] = 0;
```

```

// 将密钥字符填充到 Key 数组中，如果密钥不足，循环使用
for (int j = 0; j < 4; j++) {
    int index = (i * 4 + j) % key_len; // 使用 % key_len 来循环访问密
    钥字符

    Key[i] |= (unsigned int)(key[index]) << (24 - 8 * j);
}
}

// 设置密钥参数并进行轮密钥扩展
SetPara();
SetRoundKey(Key);

// 分块处理明文，每块 16 个字节（4 个 32 位整数）
int num_blocks = (plaintext.length() + 15) / 16; // 计算需要多少块
unsigned int Output[4 * num_blocks]; // 存储所有块的加密结果

for (int block = 0; block < num_blocks; block++) {
    // 处理每一块明文，将其转换为 4 个 32 位整数
    unsigned int Input[4] = {0};
    for (int i = 0; i < 4 && block * 16 + i * 4 < plaintext.length(); i++) {
        Input[i] = 0;
        for (int j = 0; j < 4 && block * 16 + i * 4 + j < plaintext.length(); j++)
        {
            Input[i] |= (unsigned int)(plaintext[block * 16 + i * 4 + j]) << (24
- 8 * j);
        }
    }

    // 对每块明文进行加密

```

```

        Crypt(&Output[block * 4], Input, RK, true); // 加密
    }

    // 输出所有块的密文
    cout << "密文的十六进制表示: ";
    for (int i = 0; i < 4 * num_blocks; i++) {
        printf("%08x ", Output[i]);
    }
    cout << endl;

    // 解密
    unsigned int Decrypted[4 * num_blocks];
    for (int block = 0; block < num_blocks; block++) {
        // 对每块密文进行解密
        Crypt(&Decrypted[block * 4], &Output[block * 4], RK, false); // 解密
    }

    // 输出所有块的解密结果
    cout << "解密后的明文的十六进制表示: ";
    for (int i = 0; i < 4 * num_blocks; i++) {
        printf("%08x ", Decrypted[i]);
    }
    cout << endl;

    // 将解密后的明文转换回字符串并输出
    cout << "解密后的明文: ";
    for (int block = 0; block < num_blocks; block++) {
        for (int i = 0; i < 4 && block * 16 + i * 4 < plaintext.length(); i++) {
            for (int j = 0; j < 4 && block * 16 + i * 4 + j < plaintext.length(); j++)
{

```

```

        cout << (char)((Decrypted[block * 4 + i] >> (24 - 8 * j)) & 0xFF);

    }

}

cout << endl;

return 0;

}

```

SM4 密码输出界面截屏如下：

```

请输入明文 (ASCII字符串): Hi,this is SM4 cipher!
请输入密钥 (ASCII字符串): Feistel
明文的十六进制表示: 48 69 2c 74 68 69 73 20 69 73 20 53 4d 34 20 63 69 70 68 65 72 21
密钥的十六进制表示: 46 65 69 73 74 65 6c
密文的十六进制表示: f907713d 28cc4820 d66ddb34 0e199086 8ccd0ab4 dbde8bf3 f7b5a8cb 3190ab2d
解密后的明文的十六进制表示: 48692c74 68697320 69732053 4d342063 69706865 72210000 00000000 00000000
解密后的明文: Hi,this is SM4 cipher!

```